

Degree-3 spectral refinement: research-draft verification package

Repository source: README.md

Contents

Candidate coefficient	1
What has been checked	1
What remains before public release	2
Automated verification and reproducibility	2
Typeset PDF versions of the Markdown documentation	2
Build the paper	3
Run the exact Python checks	3
Run the R cross-check	3
Current verification language	3

Author: Bruce J. Swihart

AI system disclosed: OpenAI ChatGPT (GPT-5.6 Pro), accessed August-September 2026

This is a **private, pre-release verification project** for a candidate refinement of Nishioka’s lower bound for the three-dimensional Blaschke–Lebesgue problem. The repository now contains an integrated LaTeX research draft, but it is not yet ready for public release.

Candidate coefficient

The proposed bound is

$$\text{Vol}(K) \geq \frac{\pi}{1914} \left(259 - 27\sqrt{\frac{15183}{17303}} \right) d^3 \approx 0.3836027047 d^3.$$

Nishioka’s published coefficient is $4\pi/33 \approx 0.3807991095$. The candidate is numerically weaker than the much newer HYRA claims; its possible contribution is the independent degree-3 spectral mechanism.

What has been checked

- A general degree-3 spherical harmonic is represented by a seven-parameter symmetric trace-free tensor.
- The central norm and determinant identities are integrated exactly over S^2 with SymPy rational arithmetic.
- The crucial constants are derived from polynomial identities rather than inserted as final answers.
- A second exact formulation checks the determinant identity independently.
- Deterministic numerical quadrature and random tensor tests check conventions.
- Independent Python and base-R numerical implementations both pass; see `expected/CROSS_LANGUAGE_COMPARISON.md`.
- The final coefficient is simplified and evaluated exactly.
- The noncomputational bridge from the determinant estimate to the candidate volume bound has a line-by-line draft in `proof/BRIDGE_LEMMAS.md`.
- The degree-3 tensor identity now has a human-readable derivation in `proof/DEGREE3_TENSOR_IDENTITY.md`, including a contraction-graph count for all six quartic moments.
- The contraction-graph table is independently enumerated by exact Python code; a matching base-R script is included for cross-language checking.
- An internal line-by-line audit is recorded in `proof/PROOF_AUDIT.md`.

- The complete proof has been merged into an eight-page LaTeX manuscript in `paper/degree3_spectral_refinement.tex`; the tracked PDF was built and visually inspected in this project environment.

What remains before public release

- Have the named author finish reading and approve the complete manuscript.
- Perform and record a fresh-clone reproducibility test.
- At the first public release, add the public repository URL, release version, and release date to `CITATION.cff`.
- Obtain outside mathematical review after the first public research-draft release.

The first hosted GitHub Actions run has passed all three jobs. Authorship, AI-provenance, citation, and licensing metadata are now included in the repository.

See `proof/PROOF_LEDGER.md`, `proof/CLAIM_DEPENDENCIES.md`, `proof/PROOF_AUDIT.md`, `proof/BRIDGE_LEMMAS.md`, and `proof/DEGREE3_TENSOR_IDENTITY.md`.

Automated verification and reproducibility

The workflow in `.github/workflows/verification.yml` runs on pushes to `main`, pull requests, and manual dispatch. It uses separate Ubuntu jobs to:

- install the declared Python dependencies and run every exact and numerical Python check;
- install R and run the independent pairing-count and numerical checks; and
- install Pandoc and TeX, rebuild the research paper and every Markdown-derived PDF, and preflight the generated PDF files and checksum manifest.

A green workflow run means that the posted code and document sources execute successfully in a fresh hosted environment. It does **not** constitute peer review or independent verification of the mathematical argument. The document job also checks consistency of the author name, AI-system identification, citation metadata, and licensing declarations.

- [Verification status](#)
- [Reproducibility guide](#)
- [AI assistance and provenance](#)
- [Licensing notice](#)
- [Citation metadata](#)

Typeset PDF versions of the Markdown documentation

GitHub does not consistently render the repository's single-backslash LaTeX math delimiters. Typeset PDF counterparts are tracked under `rendered/markdown/`, with the source directory structure preserved. The Markdown files remain the editable source of truth.

- [Repository overview PDF](#)
- [AI assistance and provenance PDF](#)
- [Reproducibility guide PDF](#)
- [Verification status PDF](#)
- [Cross-language comparison PDF](#)
- [Paper build notes PDF](#)
- [Bridge lemmas PDF](#)
- [Claim dependencies PDF](#)
- [Degree-3 tensor identity PDF](#)
- [Internal proof audit PDF](#)
- [Proof ledger PDF](#)

Regenerate every Markdown-derived PDF from the repository root with:

```
./build_markdown_pdfs.sh
```

The build requires Pandoc 3 or later, Python 3, and a XeLaTeX installation. The script recognizes the `\(...\)` and `\[...\]` delimiters used in the proof notes, converts the few multi-line tagged displays to valid `amsmath` environments, normalizes the PDF trailer identifiers for reproducible output, and writes a SHA-256 manifest to `rendered/markdown/SHA256SUMS.txt`.

Build the paper

The manuscript source and tracked PDF are in `paper/`. Build from the repository root with:

```
./build_paper.sh
```

The build requires `latexmk` and a standard LaTeX installation. See `paper/README.md`.

Run the exact Python checks

From the repository root:

```
python -m pip install -r requirements.txt
./run_python_checks.sh
```

The exact symbolic scripts are:

- `python/verify_degree3_identity.py`
- `python/verify_degree3_identity_alt.py`
- `python/verify_bound_arithmetic.py`
- `python/verify_pairing_counts.py`

The numerical stress test is:

- `python/numerical_stress_tests.py`

Run the R cross-check

The R scripts use base R only:

```
Rscript R/verify_pairing_counts.R
Rscript R/numerical_stress_tests.R
```

The first script independently checks the finite pairing-count table; the second is an independent numerical tensor cross-check. The general exact symbolic integration is performed by the Python/SymPy scripts.

Current verification language

Use:

AI-assisted, non-peer-reviewed research draft seeking independent mathematical verification.

Do not use:

Peer-reviewed, certified, or independently verified theorem.